

Plan de développement — Projet « Bit-Thune » (Blockchain simplifiée en C)

Équipe : Antoine Deveze • Johan Delvalle • Ethan Napoletano

1. Objectif du projet

But : implémenter en C une crypto-monnaie basée sur une blockchain centralisée et simplifiée, avec minage, transactions, économie (helicopter money + phase de marché), puis scripts de verrouillage/déverrouillage et génération de clés/adresses via OpenSSL.

2. Ce que nous allons programmer

Le projet sera structuré en modules C afin de garder un code testable et évolutif entre les parties 1, 2 et 3.

2.1 Fonctionnalités (par parties)

1. Partie 1.1 — Mise en place de la blockchain

- Structures de données : bloc, blockchain, transaction simplifiée, hashage.
- Génération des utilisateurs user1..userN.
- Calcul d'arbre de Merkle des transactions d'un bloc.
- Minage (choix du mineur au hasard, preuve de travail simplifiée si demandée par la structure).
- Vérifications de cohérence : chaînage des blocs (hash précédent), hash Merkle par bloc.
- Wallet simplifié : nom + solde.
- Transaction simple userX → userY d'un montant m.
- Affichage JSON des blocs/transactions/blockchain + affichage des wallets.
- Programme principal avec menu interactif + export de la blockchain en JSON (test via script Python fourni).

2. Partie 1.2 — Mise en place d'une économie

- Fonction « Helicopter money » : distribution initiale (coinbase → tous les users, ex. 50 000 BT), blocs minés aléatoirement, récompense de minage.
- Ajout au menu : lancer la phase helicopter money et afficher la masse monétaire en fin de boucle.

3. Partie 2 — Transactions UTXO + phase de marché

- Transactions « bitcoin-like » : listes d'inputs/outputs, timestamp, hash de transaction.
- Liste globale d'UTXO (création/suppression à chaque transaction).
- Sélection d'UTXO pour payer (stratégie greedy, inspirée du script greedy.py).
- Frais de transaction + transaction de récompense (coinbase) sans frais.
- Boucle de marché infinie : génération de transactions aléatoires, complétion par le mineur, minage, mise à jour masse monétaire, halving (division de la récompense), arrêt propre via Ctrl-C (affichage stats et reprise/stop).

4. Partie 3 — Scripts + clés/adresses

- Génération cryptographique : clé privée aléatoire, clé publique, adresse (P2PKH simplifié) via "key-sha-ripem-ad.c".
- Mécanisme de signature/vérification (Jordaney.zip).
- Mini-langage de script (pile) : DUP, HASH, EQ, VER ; exécution lock+unlock pour valider la dépense d'UTXO.
- Intégration des scripts dans les inputs/outputs et validation lors de la construction/mise à jour des UTXO.

2.2 Livrables attendus

- Code C compilable (Makefile ou commande de compilation) + README d'utilisation.
- Exécutable principal avec menu + logs lisibles.
- Export JSON valide (vérifié par le script Python).
- Jeu de tests minimal (ex : vérification hash/merkle, cohérence chaîne, cohérence UTXO, scripts).
- Rapport court : choix d'implémentation, limites, et comment rejouer une démonstration.

3. Architecture logicielle proposée

Organisation recommandée du dépôt (indicatif) :

- include/ : headers (types, constantes, prototypes).
- src/ : implémentations par module.
- tests/ : tests unitaires / scripts de validation (optionnel).
- data/ : exports JSON et exemples.

Découpage en modules :

<u>Module</u>	<u>Responsabilité principale</u>
bc_types.h / bc_types.c	Structures (Block, Tx, Input, Output, UTXO, Wallet, Blockchain) + fonctions de création/destruction.
hashing.c	(Sha256.zip + key-sha-ripem-ad.c), hash de transaction, hash de bloc.
merkle.c	Construction de l'arbre de Merkle à partir de la liste des transactions.
mining.c	Choix du mineur, preuve de travail simplifiée (si nécessaire), création bloc, récompense, halving.
utxo.c	Gestion de la liste globale UTXO (ajout/suppression/recherche), sélection greedy.
scripts_vm.c	Machine de pile et opcodes DUP/HASH/EQ/VER, exécution lock+unlock.
keys_addr.c	Génération clefs/adresses P2PKH simplifiée (key-sha-ripem-ad.c comme base).
json_io.c	Sérialisation JSON (blockchain, blocs, tx, wallets) + sauvegarde fichier.
cli.c / main.c	Menu interactif, commandes, ctrl-C (ctrl_c.c) et boucle de marché.

4. Répartition des tâches (équipe de 3)

Répartition proposée pour travailler en parallèle tout en gardant des interfaces claires entre modules :

Membre

Sous-systèmes & tâches

Antoine Deveze

- Socle blockchain (structures bloc/chain/tx simplifiée) + vérifications cohérence (hash chaînage, Merkle).
- JSON export + menu interactif (partie 1) + intégration du validateur testjson.py.
- Intégration finale et packaging (README, Makefile, démos).

Johan Delvalle

- Économie : helicopter money, masse monétaire, récompense, halving.
- Phase de marché (boucle infinie, génération tx aléatoires) + gestion Ctrl-C propre (ctrl_c.c).
- UTXO : liste globale + mise à jour (ajout/suppression) + calcul frais.

Ethan Napoletano

- Crypto OpenSSL : génération clés/adresses + hash SHA256/RIPEMD160 (key-sha-ripem-ad.c, Sha256.zip).
- Signatures / vérification (Jordaney.zip) et intégration P2PKH simplifiée.
- VM de scripts (DUP/HASH/EQ/VER) + validation lock/unlock sur UTXO (partie 3).

Points de synchronisation :

- Définir tôt les structures communes (Tx/Input/Output/UTXO/Block) et les fonctions d'allocation/libération pour éviter les fuites mémoire.
- Définir un format JSON stable dès la partie 1 (mêmes champs, extensible).
- Tester régulièrement la cohérence chaîne+UTXO après chaque ajout de bloc.

5. Échéancier (proposition)

Organisation sur 6 semaines à partir du 3 mars 2026, avec une livraison interne chaque fin de semaine. Ajustable selon les dates de CC2/CC3/CC4.

<u>Semaine</u>	<u>Dates</u>	<u>Objectifs techniques</u>	<u>Livrable / jalon</u>
S1	09/03–11/03/2026	Lecture sujet + choix structures + squelette projet + compilation OpenSSL	Repo initial, build OK, types principaux définis
S2	12/03–27/03/2026	Partie 1.1 : merkle, hash blocs/tx, minage, vérif cohérence, JSON export, menu	Démo : création chaîne + tx simple + export JSON validé
S3	28/03–13/04/2026	Partie 1.2 : helicopter money + wallets + masse monétaire + récompense	Démo : distribution + blocs minés + stats masse monétaire
S4	16/04–03/05/2026	Partie 2 : Tx inputs/outputs + UTXO globale + greedy + frais + boucle marché (sans scripts)	Démo : marché tourne, UTXO cohérents, Ctrl-C affiche stats
S5	04/05–24/05/2026	Partie 3 : clés/adresses + signatures + scripts VM + validation lock/unlock	Démo : tx refusées si script invalide, adresses vérifiées
S6	25/05–07/06/2026	Stabilisation : tests, mémoire, robustesse, doc, scénarios de démonstration	Release candidate : archive + rapport + jeu de démos

6. Risques et parades

<u>Risque</u>	<u>Parade</u>
Gestion mémoire (listes chaînées, allocations en cascade)	Convention de propriété des pointeurs + fonctions <code>free_*</code> + tests valgrind.
Crypto/OpenSSL (API, formats binaires/hex/base58)	Prototyper tôt avec <code>key-sha-ripem-ad.c</code> et comparer avec outils web (sha256, validateur d'adresse).
Cohérence UTXO vs blockchain	Fonction d'audit qui reconstruit l'état UTXO depuis la chaîne pour comparer à la liste globale.
Boucle de marché / interruptions	Utiliser un handler Ctrl-C minimal (flag) + arrêt propre au point sûr (fin d'itération).

Annexe : outils fournis / liens de vérification (à utiliser pendant le dev)

- Test adresses/clefs : [rfctools.com bitcoin-address-test-tool](https://rfctools.com/bitcoin-address-test-tool)
- Tutoriel ECDSA : [learnmeabitcoin.com \(ECDSA/sign\)](https://learnmeabitcoin.com/ECDSA/sign)
- Exercices OpenSSL ECDSA : rjordaney.is
- SHA256 online : emn178.github.io
- Scripts Python fournis : `testjson.py` (validation JSON), `greedy.py` (sélection greedy) + `res.json` (exemple).